

Special Application: Face Recognition

What is Face Recognition?

Face verification vs. face recognition

→ Verification

- Input image, name/ID
- Output whether the input image is that of the claimed person

1:1 99%

→ Recognition

- Has a database of K persons
- Get an input image
- Output ID if the image is any of the K persons (or “not recognized”)

1:K
K=1

Andrew Ng

that K is equal to 100 in a recognition system.

Face Recognition

Course Overview

This week focuses on two important applications of Convolutional Neural Networks (CNNs):

- **Face Recognition**
- **Neural Style Transfer**

The programming exercises include implementing neural style transfer to generate artistic images.

Face Recognition Example

A face recognition system can replace traditional ID-card authentication.

Example workflow:

```
Person approaches entrance
↓
Face Recognition System
↓
Identity Verified
↓
Access Granted
```

Benefits:

- Contactless authentication
- Fast user identification
- Improved convenience

Liveness Detection

The demonstration also includes **liveness detection**.

Purpose

Determine whether the presented face belongs to:

- A **live person**

- A **fake presentation** (e.g., a printed photo)

Example:

- Using another person's ID card with a printed face is rejected.
- Using the actual person's live face is accepted.

Liveness detection can be trained as a supervised learning problem:

Live Human
vs.
Not Live Human

Face Verification

Definition

Face verification determines whether an input face matches a claimed identity.

Input:

- Face image
- Claimed identity (name or ID)

Output:

Is this person who they claim to be?

YES / NO

This is known as a **one-to-one matching** problem.

Face Recognition

Definition

Face recognition determines **which person** in a database matches the input face.

Input:

- Face image
- Database of known identities

Output:

Who is this person?

This is a **one-to-many matching** problem.

Verification vs. Recognition

Face Verification	Face Recognition
One-to-one comparison	One-to-many comparison
Verify a claimed identity	Identify a person from a database
Simpler problem	More difficult problem

Why Recognition Is Harder

Suppose a verification system has:

99% accuracy

If the recognition database contains:

K = 100 people

the system must compare the input against many possible identities.

As the number of candidates increases:

- The probability of making at least one incorrect comparison also increases.
- Recognition therefore requires much higher verification accuracy.

For a database of around **100 people**, the verification model may need accuracy around:

99.9% or higher

to achieve reliable recognition performance.

Building a Recognition System

A common strategy is:

High-Accuracy Face Verification
↓
Face Recognition System

Face verification serves as the fundamental building block for face recognition.

Key Takeaways

- This week introduces **Face Recognition** and **Neural Style Transfer**.
- Face recognition systems often incorporate **liveness detection** to prevent spoofing attacks.
- **Face Verification** checks whether an image matches a claimed identity (**one-to-one**).
- **Face Recognition** identifies a person from a database (**one-to-many**).
- Recognition is more challenging because matching errors accumulate as the database grows.
- High-performance recognition systems are typically built upon highly accurate face verification models.
- The next topic introduces **one-shot learning**, a key challenge in face verification.

One Shot Learning

One-shot learning

Learning from one example to recognize the person again

what you're going to do instead is learn a similarity function.

Andrew Ng

One-Shot Learning in Face Recognition

What is One-Shot Learning?

One-shot learning refers to the ability to recognize a person after seeing **only one example** of that person's face.

This is a fundamental challenge in face recognition because, in many real-world applications, only a single reference image is available for each person.

Example

Suppose the employee database contains one image for each person:

- Person A
- Person B
- Person C
- Person D

When someone arrives:

- If the face matches one of the stored images, access should be granted.
- If the face does not match anyone in the database, the system should reject it.

The model must recognize each person despite having seen only one training image.

Why is It Difficult?

Traditional deep learning models usually require **many training examples** per class.

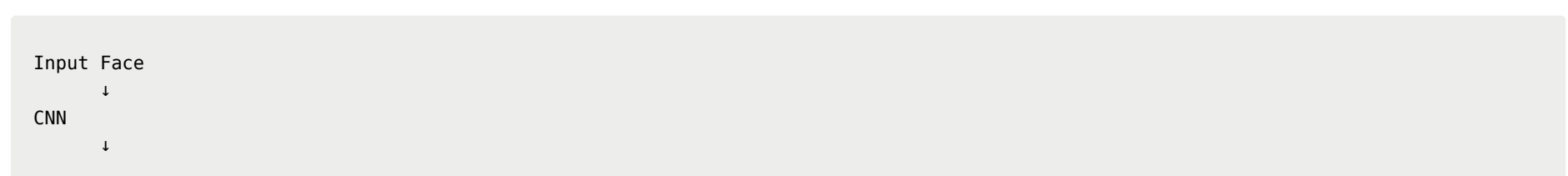
With only one image per person:

- The model cannot learn robust class-specific features.
- Generalization becomes difficult.

This makes standard supervised classification unsuitable.

Naive Approach

One possible solution is to train a CNN classifier:



```
Softmax
  ↓
Person A
Person B
Person C
Person D
Unknown
```

If there are **4 registered people**, the Softmax layer has:

5 outputs

- Person A
- Person B
- Person C
- Person D
- None of the above

Problems with the Softmax Approach

1. Insufficient Training Data

Only one image per person is usually not enough to train a robust classifier.

2. Poor Scalability

Whenever a new employee joins:

```
Old Output:
4 Persons + Unknown

↓

New Output:
5 Persons + Unknown
```

The output layer changes.

As a result, the CNN must be retrained whenever the database changes.

This is impractical for real-world face recognition systems.

Desired Solution

Instead of learning to classify a fixed set of identities, the model should learn:

▮ **Whether two face images belong to the same person.**

This approach:

- Works with one reference image.
- Easily supports new identities.
- Does not require retraining when new people are added.

This forms the basis of modern face verification systems.

Key Takeaways

- **One-shot learning** means recognizing a person from only **one reference image**.
- Most face recognition systems require one-shot learning because only one image per person is typically available.
- A standard CNN with a Softmax classifier is not suitable because:

- It requires many training examples.
- It does not scale when new identities are added.
- Modern face recognition systems instead learn to compare two face images rather than classify a fixed set of identities.

Learning a “similarity” function

→ $d(\text{img1}, \text{img2}) = \text{degree of difference between images}$

If $d(\text{img1}, \text{img2}) \leq \tau$ “same”
 $> \tau$ “different” } Verification.

$d(\text{img1}, \text{img2})$

you can add a fifth person to your database, and it just works fine.

Andrew Ng

Learning a Similarity Function

Core Idea

Instead of training a classifier to recognize a fixed set of identities, face verification learns a **similarity function**:

```
d(Image1, Image2)
```

The function measures how different two face images are.

- **Small value** → same person
- **Large value** → different people

Decision Rule

Given two face images:

```
d(Image1, Image2)
```

Compare the output with a threshold:

```
If d < τ
  → Same person

If d ≥ τ
  → Different people
```

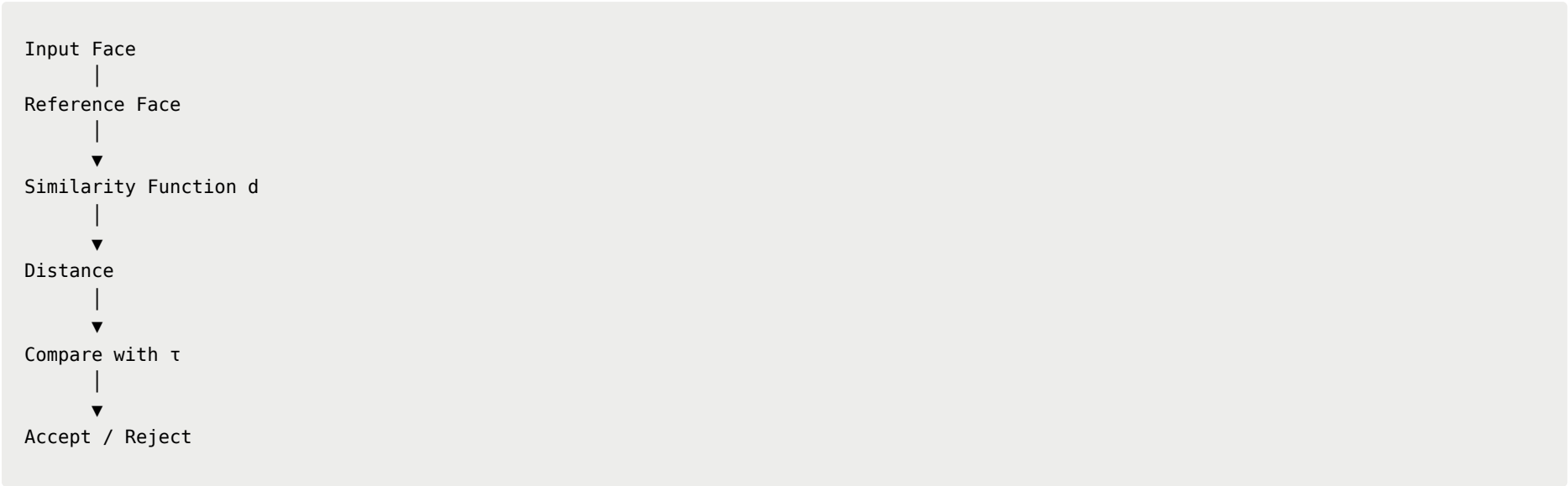
where:

- τ (tau) is a predefined threshold (hyperparameter).

Face Verification

For verification, the system compares:

- Input face
- Claimed identity's reference image



The decision depends only on whether the distance is below the threshold.

Face Recognition Using Similarity

To perform recognition:

1. Compare the input face with every face stored in the database.
2. Compute the similarity score for each comparison.
3. Select the smallest distance.

Example:

```
Input Face
↓
d(Input, Person A) = 10
d(Input, Person B) = 0.3
d(Input, Person C) = 8
d(Input, Person D) = 12
```

Since the smallest distance is for **Person B**, the system predicts that the input belongs to Person B.

Unknown Person Detection

If the input person is **not** in the database:

```
d(Input, Person A) = 9
d(Input, Person B) = 11
d(Input, Person C) = 8
d(Input, Person D) = 10
```

All distances are large.

Therefore, the system predicts:

```
Unknown Person
```

Why This Solves One-Shot Learning

The network learns **how to compare faces**, rather than memorizing identities.

As a result:

- Only one reference image per person is needed.
- New people can be added by simply storing their reference image.
- The neural network does **not** need to be retrained when the database changes.

Advantages

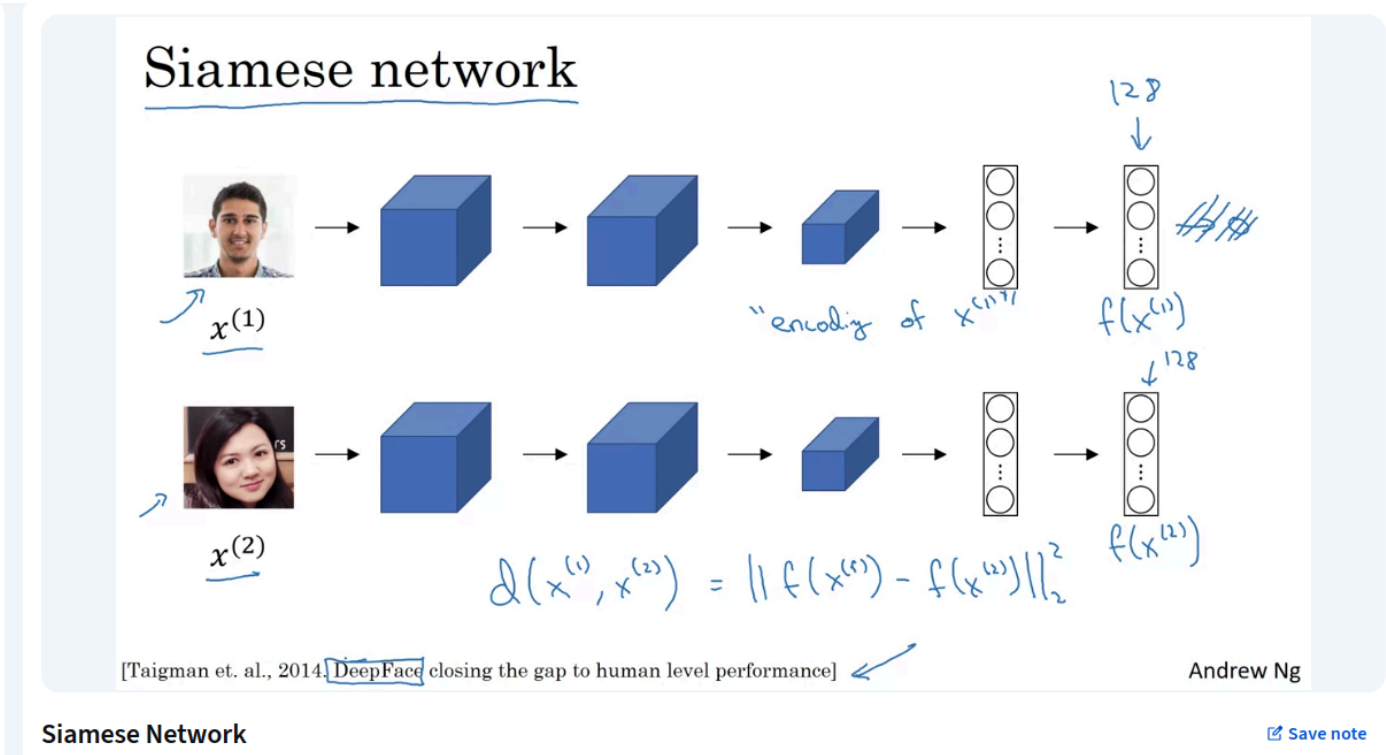
Compared with a Softmax classifier:

Softmax Classification	Similarity Function
Fixed number of identities	Supports any number of identities
Retraining required for new people	Simply add a new reference image
Needs many examples per class	Works with one-shot learning
Learns class labels	Learns face similarity

Key Takeaways

- Face verification learns a **similarity (distance) function** `d(Image1, Image2)`.
- A **small distance** indicates the same person, while a **large distance** indicates different people.
- A threshold `τ` determines whether two faces match.
- Face recognition is performed by comparing the input face against every stored reference image and selecting the smallest distance.
- This approach naturally supports **one-shot learning** and allows new identities to be added without retraining the model.

Siamese Network



Goal of learning

Parameters of NN define an encoding $f(x^{(i)})$ 128

Learn parameters so that:

If $x^{(i)}, x^{(j)}$ are the same person, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is small.

If $x^{(i)}, x^{(j)}$ are different persons, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is large.

}

Andrew Ng

Siamese Network

[Save note](#)

Siamese Network for Face Verification

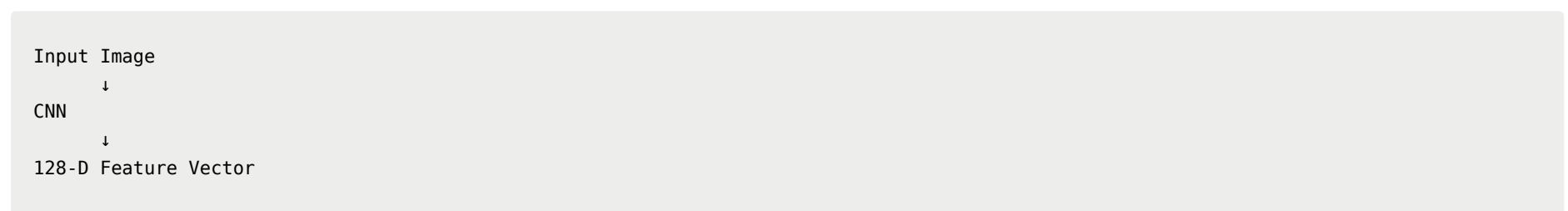
Motivation

To solve face verification, instead of classifying identities directly, the model learns a **similarity function** by encoding each face into a feature vector.

The similarity between two faces is measured by comparing their encodings.

Face Encoding

A CNN processes an input face image and outputs a fixed-length feature vector.



The encoding is denoted as:

$f(x)$

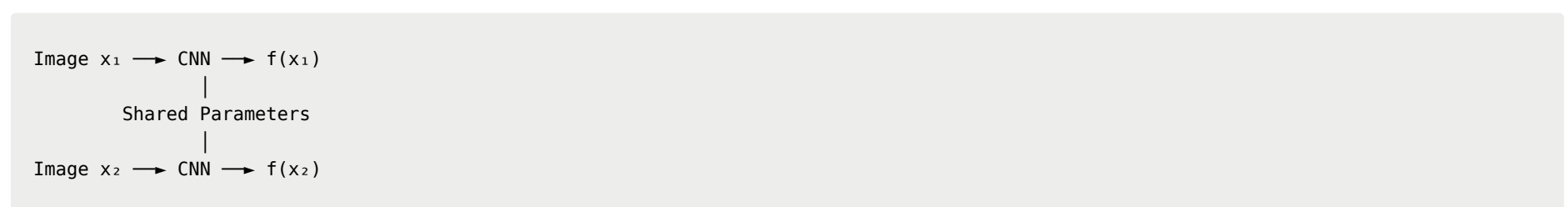
where:

- x : input face image
- $f(x)$: 128-dimensional embedding (feature representation)

This embedding captures the important characteristics of the person's face.

Siamese Network Architecture

A Siamese Network consists of **two identical CNNs**.



Key property:

- Both networks share **exactly the same parameters (weights)**.
- The same feature extractor is applied to both input images.

Similarity Measure

After computing the two embeddings, their distance is calculated.

$$d(x_1, x_2) = ||f(x_1) - f(x_2)||$$

where:

- $f(x_1)$ and $f(x_2)$ are the learned embeddings.
- $|| \cdot ||$ denotes a vector norm (distance).

Interpretation of Distance

Same Person

If the two images belong to the same person:

$$d(x_i, x_j) \rightarrow \text{Small}$$

Their embeddings should be close together.

Different People

If the two images belong to different people:

$$d(x_i, x_j) \rightarrow \text{Large}$$

Their embeddings should be far apart.

Training Objective

The CNN parameters define the embedding function:

$$f(x)$$

Training aims to learn embeddings such that:

Positive Pair

Same Person
↓
Small Distance

Negative Pair

Different People
↓
Large Distance

By optimizing the CNN parameters, the embeddings become increasingly discriminative.

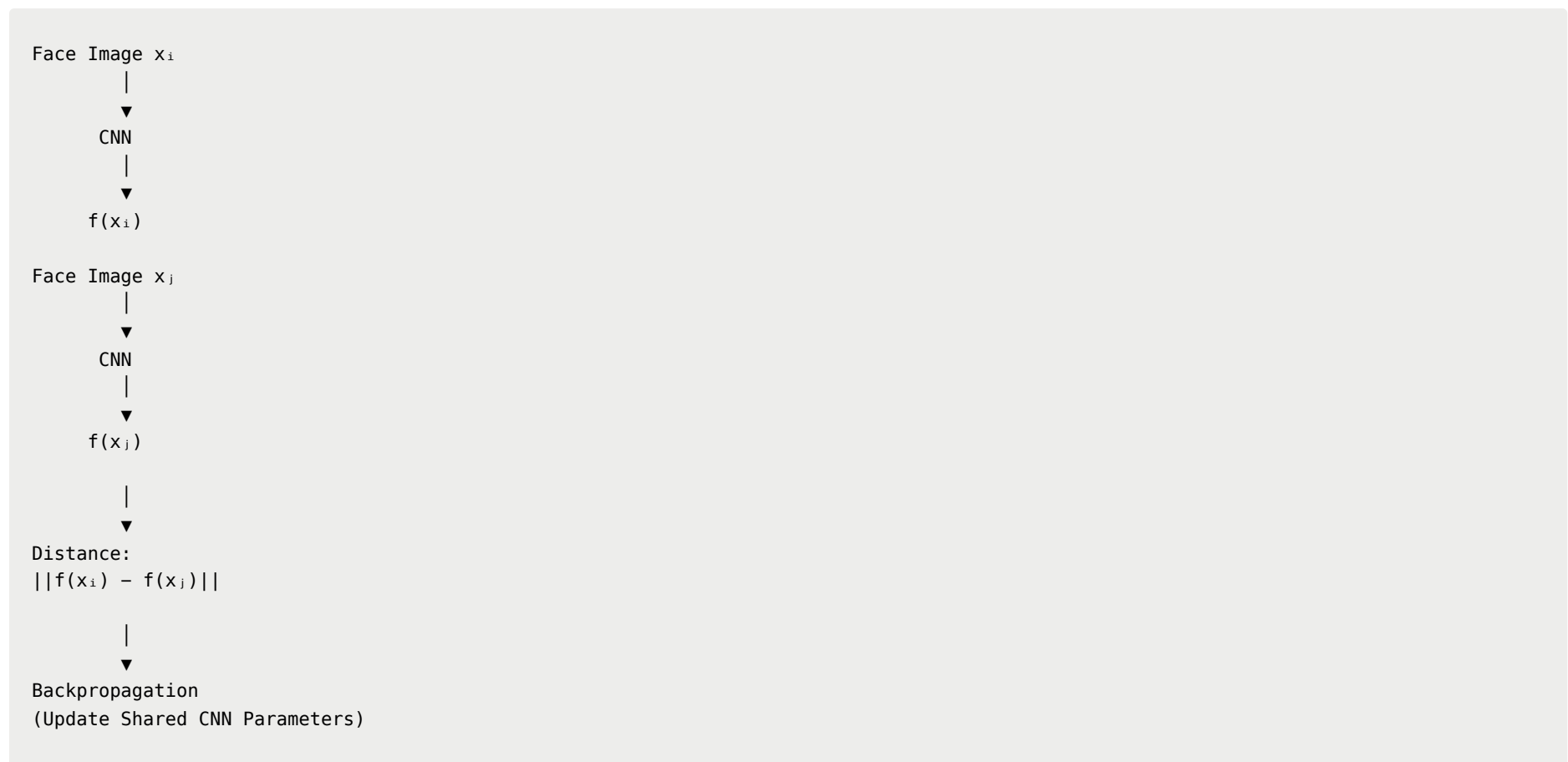
Why Shared Parameters?

Using identical weights ensures:

- Both images are mapped into the **same embedding space**.
- Distances between embeddings are meaningful.

- The comparison is consistent regardless of input order.

Training Process



The network learns embeddings that naturally separate different identities while grouping images of the same identity.

Next Step

To train the Siamese Network, we need an objective function that encourages:

- Small distances for positive pairs.
- Large distances for negative pairs.

This is achieved using the **Triplet Loss**, which will be introduced next.

Key Takeaways

- A **Siamese Network** consists of two identical CNNs with **shared parameters**.
- Each CNN maps an input face to a **128-dimensional embedding** $f(x)$.
- Face similarity is measured using the embedding distance:

$$d(x_1, x_2) = ||f(x_1) - f(x_2)||$$

- Images of the **same person** should have **small embedding distances**.
- Images of **different people** should have **large embedding distances**.
- The CNN is trained through **backpropagation** to learn an embedding space suitable for face verification.
- The **Triplet Loss** provides the training objective for learning these embeddings.

Triplet Loss

Learning Objective



Anchor Positive

A P

$d(A, P) = 0.5$



Anchor Negative

A N

$d(A, N) = 0.7$

Want: $\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$

$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$

margin

$f(\text{img}) = \vec{0}$

[Schroff et al., 2015, FaceNet: A unified embedding for face recognition and clustering] 1x Andrew Ng

Triplet Loss

Purpose

Triplet Loss trains a Siamese Network to learn face embeddings such that:

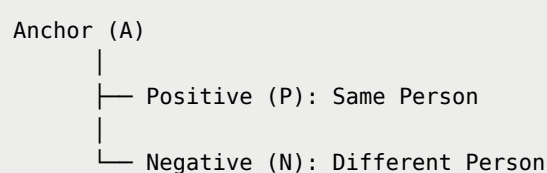
- Images of the **same person** are close together.
- Images of **different people** are far apart.

Instead of comparing only two images, Triplet Loss considers **three images simultaneously**.

Triplet Components

Each training example consists of:

- **Anchor (A)**: reference image
- **Positive (P)**: image of the same person
- **Negative (N)**: image of a different person



Desired Property

The learned embeddings should satisfy:

$$\text{Distance}(A, P) < \text{Distance}(A, N)$$

where:

- $\text{Distance}(A, P)$ measures similarity between the anchor and the positive example.
- $\text{Distance}(A, N)$ measures similarity between the anchor and the negative example.

The positive pair should always be closer than the negative pair.

Distance Function

Distances are computed using the learned embeddings:

$$d(A, P) = ||f(A) - f(P)||^2$$

$$d(A, N) = ||f(A) - f(N)||^2$$

where:

- $f(x)$ is the embedding generated by the CNN.
- $|| \cdot ||^2$ denotes the squared Euclidean distance.

Basic Constraint

The desired relationship is:

$$||f(A) - f(P)||^2 \leq ||f(A) - f(N)||^2$$

Equivalently,

$$||f(A) - f(P)||^2 - ||f(A) - f(N)||^2 \leq 0$$

Why This Is Not Enough

The network could satisfy this constraint with a **trivial solution**.

Example:

$$f(x) = 0$$

for every image.

Then:

$$\text{Distance}(A, P) = 0$$

$$\text{Distance}(A, N) = 0$$

The inequality is satisfied, but the embeddings contain no useful information.

Another trivial solution is assigning **the same embedding vector to every image**.

Margin

To prevent trivial solutions, Triplet Loss introduces a **margin** α (alpha).

The new constraint becomes:

$$||f(A) - f(P)||^2 - ||f(A) - f(N)||^2 + \alpha \leq 0$$

where:

- α is a positive hyperparameter.
- Typical example:

$$\alpha = 0.2$$

Why Margin Helps

Suppose:

$$d(A, P) = 0.5$$

Without a margin:

$$d(A, N) = 0.51$$

technically satisfies:

$$d(A, P) < d(A, N)$$

but the separation is extremely small.

With:

$$\alpha = 0.2$$

the network must satisfy approximately:

$$\begin{aligned} d(A, N) \\ \geq \\ d(A, P) + 0.2 \end{aligned}$$

Example:

$$0.5 + 0.2 = 0.7$$

So the negative example should be at least **0.7** away.

The network can satisfy this by:

- decreasing $d(A, P)$,
- increasing $d(A, N)$,
- or both.

Training Objective

Triplet Loss encourages the embedding space to satisfy:

Anchor
|
Positive → Close
Negative → Far Away

Over many triplets, the CNN learns an embedding space where:

- Same identities cluster together.

- Different identities are well separated.

Key Takeaways

- Triplet Loss trains the network using **three images: Anchor, Positive, and Negative**.
- The goal is to make:
 - $d(A, P)$ **small**
 - $d(A, N)$ **large**
- Distances are computed using the learned embeddings $f(x)$.
- Simply enforcing $d(A, P) < d(A, N)$ leads to trivial solutions.
- A **margin** α requires the negative example to be **significantly farther** than the positive example.
- The margin improves the discriminative power of the learned embedding space and enables robust face verification.

Loss function

Given 3 images A, P, N :

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

$J = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)})$

Training set: 10k pictures of 1k persons

Now, how do you actually

Andrew Ng

Choosing the triplets A, P, N

During training, if A, P, N are chosen randomly,
 $d(A, P) + \alpha \leq d(A, N)$ is easily satisfied.

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$$

Choose triplets that're "hard" to train on.

$$\mathcal{L}(A, P) + \alpha \leq \mathcal{L}(A, N)$$

$$\mathcal{L}(A, P) \approx \mathcal{L}(A, N)$$

how algorithms are often

Andrew Ng

Triplet Loss Function

Loss for One Triplet

A training example consists of three images:

- **Anchor (A)**
- **Positive (P)**: same identity as the anchor

- **Negative (N)**: different identity

The Triplet Loss for one triplet is:

$$L(A, P, N) = \max(||f(A) - f(P)||^2 - ||f(A) - f(N)||^2 + \alpha, 0)$$

where:

- $f(x)$ is the learned embedding.
- α is the **margin** hyperparameter.

Interpretation

The objective is:

$$d(A, P) + \alpha \leq d(A, N)$$

This means:

- The positive example should be closer to the anchor.
- The negative example should be at least **α farther away**.

Why Use $\max(\cdot, 0)$?

The loss is defined as:

$$\max(\text{expression}, 0)$$

Case 1: Constraint Satisfied

If

$$\begin{aligned} d(A, P) + \alpha \\ \leq \\ d(A, N) \end{aligned}$$

then

$$\text{Loss} = 0$$

The network already separates the embeddings sufficiently.

No further optimization is needed for this triplet.

Case 2: Constraint Violated

If

$$\begin{aligned} d(A, P) + \alpha \\ > \\ d(A, N) \end{aligned}$$

then

$$\text{Loss} > 0$$

Gradient descent updates the network to:

- decrease $d(A, P)$

- increase $d(A, N)$

until the margin constraint is satisfied.

Overall Cost Function

The total training objective is the sum of the losses over all triplets:

$$J = \sum L(A, P, N)$$

Gradient descent minimizes this objective through backpropagation.

Training Data Requirements

Unlike inference, **training requires multiple images of each identity**.

Example:

```
10,000 images
1,000 identities
≈10 images per identity
```

Multiple images are necessary to construct:

- Anchor–Positive pairs
- Anchor–Negative pairs

After training, however, the learned model can perform **one-shot learning**, requiring only one reference image per person during deployment.

Choosing Training Triplets

Random Triplets

Randomly selecting triplets is usually ineffective.

Most random triplets already satisfy:

$$d(A, P) \ll d(A, N)$$

Their loss is already zero.

These examples contribute almost no useful gradient.

Hard Triplets

Training focuses on **hard triplets**.

A hard triplet is one where:

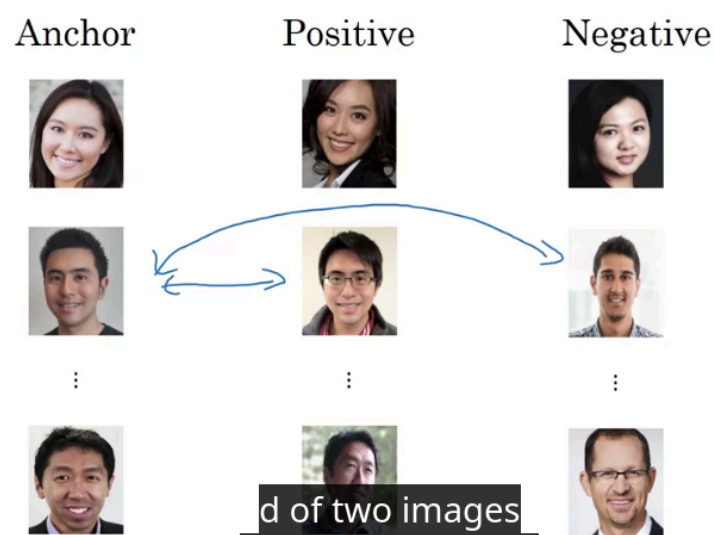
$$\begin{aligned} d(A, P) \\ \approx \\ d(A, N) \end{aligned}$$

The network must actively learn to:

- bring the anchor closer to the positive.
- push the negative farther away.

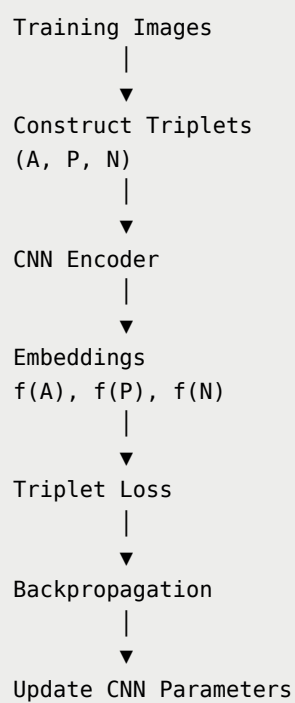
These triplets provide much stronger learning signals and improve training efficiency.

Training set using triplet loss



Andrew Ng

Training Pipeline



The learned embedding space gradually satisfies:

- Same identities → nearby embeddings.
- Different identities → well-separated embeddings.

Practical Considerations

Large-scale commercial face recognition systems are typically trained on extremely large datasets:

- Millions of images are common.
- Some systems are trained with **tens or even hundreds of millions** of images.

Because collecting datasets of this size is difficult, practitioners often use **pretrained face recognition models** instead of training from scratch.

Key Takeaways

- Triplet Loss is computed over **Anchor-Positive-Negative** triplets.
- The loss is:

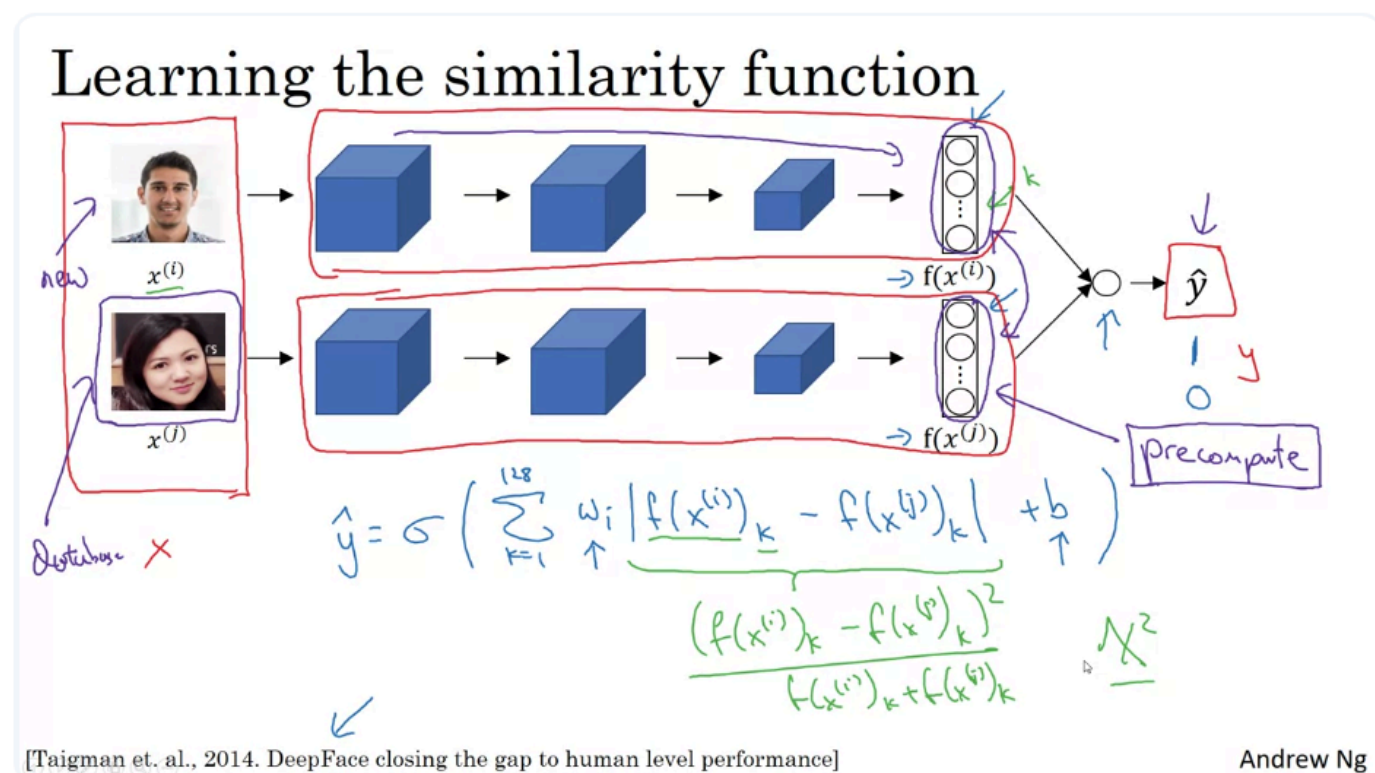
$$\max(d(A, P) - d(A, N) + \alpha, 0)$$

- The loss becomes **zero** once the margin constraint is satisfied.
- Training requires **multiple images per identity** to construct valid triplets.
- **Hard triplets** (where positive and negative distances are similar) are much more informative than random triplets.
- Gradient descent learns an embedding space where:
 - faces of the **same person** are close together,
 - faces of **different people** are separated by at least the margin α .
- In practice, face recognition models are often initialized from **large pretrained models** because they require massive datasets to train effectively.

Face Verification and Binary Classification

Andrew should have write the prediction like that:

$$y_{hat} = \sigma \left(\sum_{k=1}^{128} w_k |f(x^{(i)})_k - f(x^{(j)})_k| + b \right)$$



Face Verification and Binary Classification

[Save note](#)

Siamese Network with Binary Classification

Alternative to Triplet Loss

Besides **Triplet Loss**, a Siamese Network can also be trained as a **binary classification problem**.

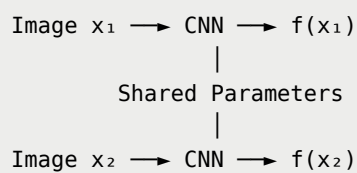
Instead of learning with triplets **(Anchor, Positive, Negative)**, the model learns from **pairs of images**.

Each pair is labeled as:

- **1** → Same person
- **0** → Different people

Network Architecture

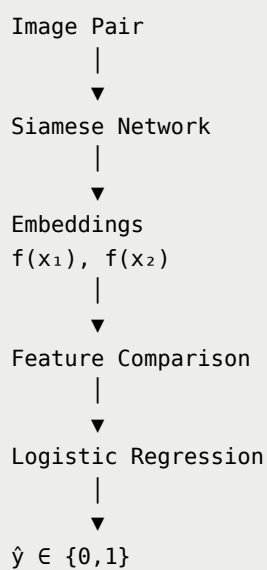
Two identical CNNs encode the input images.



Both CNNs share the same weights and produce embeddings for the two input images.

Binary Classification

After obtaining the embeddings, a **Logistic Regression** layer predicts whether the two images belong to the same person.



where:

- $\hat{y} = 1$ → Same person
- $\hat{y} = 0$ → Different people

Feature Construction

Instead of directly using the embeddings, the model computes their **element-wise absolute difference**:

$$|f(x_1) - f(x_2)|$$

If the embedding dimension is **128**, this produces **128 comparison features**.

These features are then fed into Logistic Regression.

Logistic Regression

The prediction is computed as:

$$\hat{y} = \sigma(W^T |f(x_1) - f(x_2)| + b)$$

where:

- σ is the sigmoid function.
- W and b are learnable Logistic Regression parameters.

The network is trained using standard binary classification.

Alternative Similarity Measure

Instead of the absolute difference, another commonly used feature is the **Chi-square similarity**:

$$\frac{(f(x_1)_k - f(x_2)_k)^2}{f(x_1)_k + f(x_2)_k}$$

$$f(x_1)_k + f(x_2)_k$$

computed for each embedding dimension k.

This provides another way to compare two embeddings before classification.

Training Data

Unlike Triplet Loss, training uses **pairs of images**.

Positive pair:

Image A

Image B

Label = 1

Both images belong to the same person.

Negative pair:

Image A

Image C

Label = 0

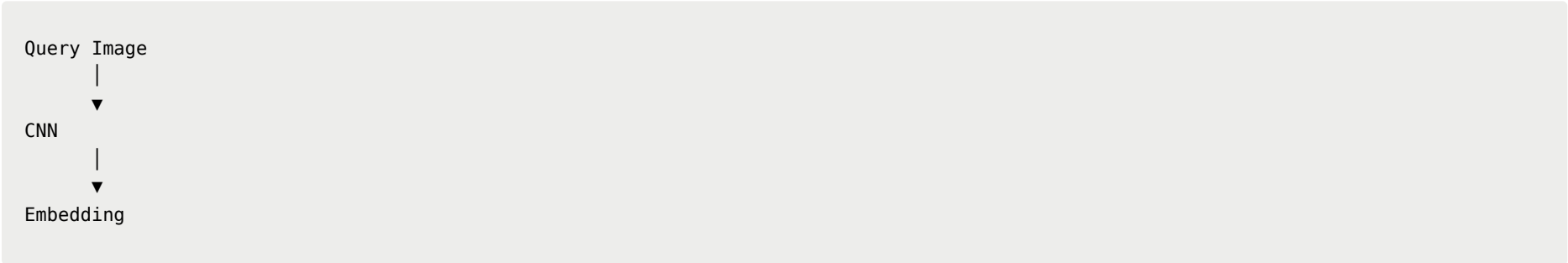
The images belong to different people.

The Siamese Network and Logistic Regression layer are trained jointly using backpropagation.

Deployment Optimization

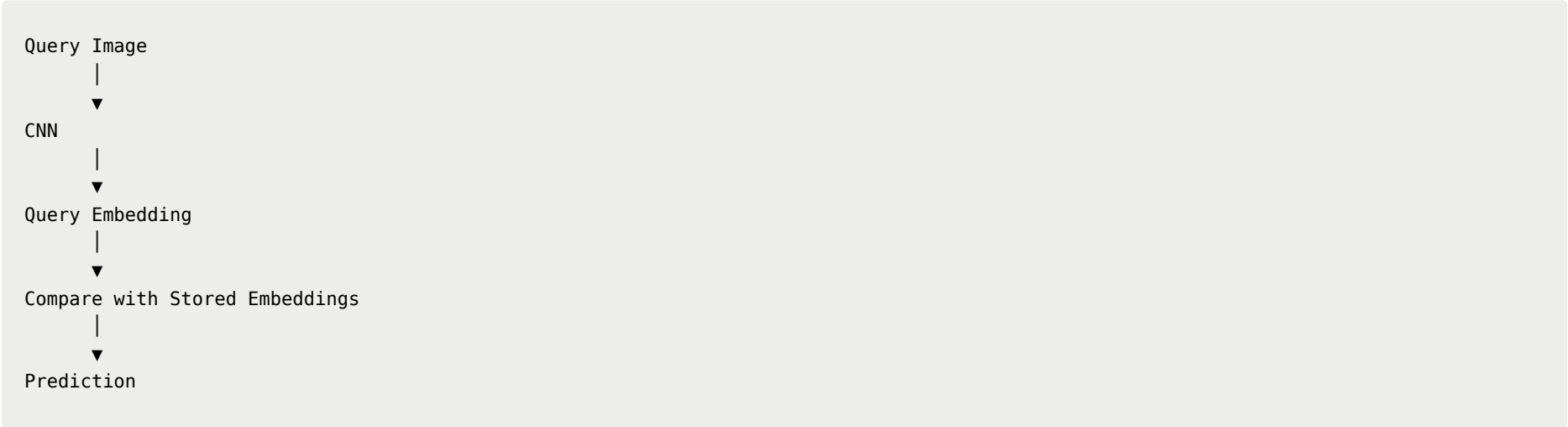
A useful optimization is **precomputing embeddings** for all images stored in the database.

Instead of computing both embeddings every time:



the database embeddings are computed **once** and stored.

During verification:



This significantly reduces computation, especially for large databases.

Triplet Loss vs. Binary Classification

Triplet Loss	Binary Classification
Uses triplets (A, P, N)	Uses image pairs
Learns embedding distances directly	Learns same/different classification
Margin-based objective	Binary cross-entropy objective
Requires hard triplet mining	Simpler pair generation

Both methods train a Siamese Network to produce meaningful face embeddings.

Key Takeaways

- A Siamese Network can be trained as a **binary classifier** instead of using Triplet Loss.
- Training samples consist of **pairs of images** labeled as **same person (1)** or **different people (0)**.
- The network computes embeddings $f(x_1)$ and $f(x_2)$, then compares them using the **element-wise absolute difference**.
- A **Logistic Regression** layer predicts whether the two images belong to the same identity.
- An alternative comparison feature is the **Chi-square similarity**.
- During deployment, **precomputing database embeddings** greatly improves efficiency because only the query image needs to pass through the CNN.

Face verification supervised learning

x	y	
	1	"Same"
	0	"Different"
	0	
	1	

the neural network to train the scientists that were using backpropagation.

Face Verification and Binary Classification

[Save note](#)